Python in Data Science: Data Manipulation

Objective

Objective

We explore the libraries most often used in Data Science for real analytical work.

- Analyzing — compute statistics and summaries over large datasets.
- Cleaning — handle missing values, wrong types, and messy rows.
- Exploring — slice, filter, and group data to find patterns.
- Manipulating — reshape, combine, and derive new columns.



CORE LIBRARIES FOR DATA MANIPULATION IN PYTHON

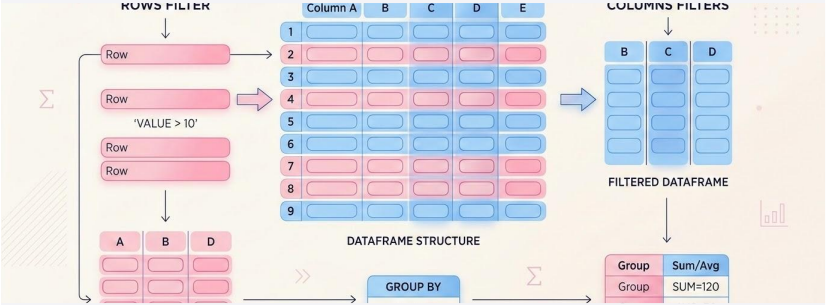
Introduction to NumPy & Pandas

01 NumPy
NUMERICAL PYTHON



- The fundamental package for scientific computing in Python.
- Provides high-performance N-dimensional array objects (ndarray).
- Offers fast vectorized operations, linear algebra, and random number generation.

02 Pandas
DATA ANALYSIS & MANIPULATION



- Built on top of NumPy for powerful data analysis and manipulation.
- Introduces intuitive DataFrame and Series data structures.
- Essential for cleaning, transforming, aggregating, and analyzing structured data.

Lesson 1 · NumPy

NUMERICAL PYTHON

The foundation of scientific computing in Python

NumPy Facts

AT A GLANCE

01

2005

First released — now the backbone of the scientific Python stack.

02

BSD

Permissive open-source license, free for commercial use.

03

Python / C

Python interface with a C core for near-native speed.

04

600M+ / yr

Downloads per year — one of Python's most-used packages.

Why NumPy

The fundamental array library

- NumPy gives fast, multi-dimensional array objects plus math tools.
- Foundation of SciPy, Pandas, Matplotlib, scikit-learn.
- Up to 100× faster than native Python lists.
- Vectorized operations — no slow Python for-loops.
- Memory-efficient: C-contiguous typed arrays.

numpy_intro.py

```python
import numpy as np

# NumPy version
print(np.__version__)      # '1.26.4'

# Quick test
a = np.array([1, 2, 3, 4, 5])
print(a * 2)               # [ 2  4  6  8 10]
```

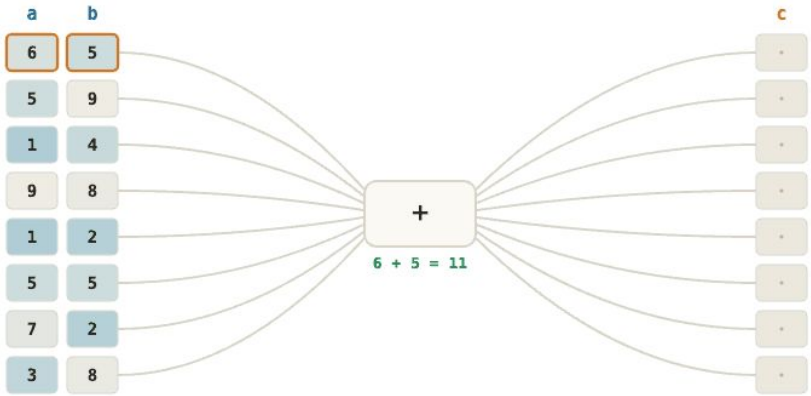
Python Loop

High control — you decide exactly what happens at each step

```
1  lst1 = [1,2,3,4,5]
2  lst2 = [6,7,8,9,10]
3  total = 0
4  for val1, val2 in zip(lst1, lst2):
5      total = total + (val1+val2)
6  print(total)
```

[3] ✓ 0.0s

```
55
```



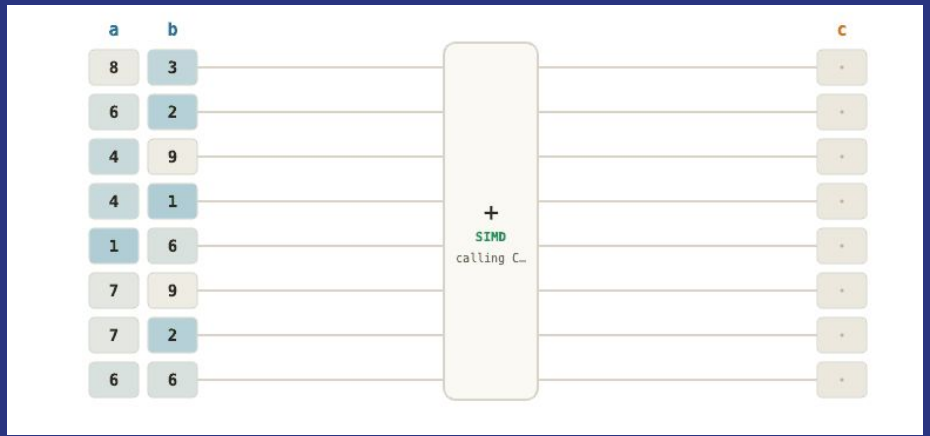
Python Numpy

Up to ~100× faster — operations run in optimized C language, working on the whole array at once rather than step-by-step.

```
1  import numpy as np
2
3  lst1 = [1,2,3,4,5]
4  lst2 = [6,7,8,9,10]
5
6  print(np.sum(lst1) + np.sum(lst2))
```

[2] ✓ 0.0s

```
55
```



Create 1D Array

One dimension array (1D)

```
mylist = [1,2,3]
x = np.array(mylist)
x
```

```
array([1, 2, 3])
```

```
y = np.array([3,4,5])
y
```

```
array([3, 4, 5])
```

```
y.shape
```

```
(3,)
```

Create 2D Array

Two dimension array (2D)

```
m = np.array([ [1,2,3], [4,5,6], [7,8,9] ])
m
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

```
m.shape
```

```
(3, 3)
```

The ndarray

One typed, fixed-size object

- .shape — tuple of dimensions, e.g. (2, 3).
- .dtype — element type, e.g. float32, int64.
- .ndim — number of axes;
- .size — total elements.
- .itemsize — bytes per element; .strides — step per axis.

ndarray.py

```
a = np.array([[1, 2, 3],
              [4, 5, 6]], dtype=np.float32)

print(a.shape)     # (2, 3)
print(a.ndim)      # 2
print(a.dtype)     # float32
print(a.size)      # 6
print(a.itemsize)  # 4 bytes
```

Lesson 1.1: Numpy - Operations

With Number

With number

```
n = np.array([1,2,3])
n
```

```
array([1, 2, 3])
```

```
n + 1
```

```
array([2, 3, 4])
```

```
n - 2
```

```
array([-1,  0,  1])
```

```
n / 3
```

```
array([0.33333333, 0.66666667, 1.        ])
```

```
n ** 2
```

```
array([1, 4, 9])
```

With array

With array

```
n = np.array([1,2,3])
n
```

```
array([1, 2, 3])
```

```
m = np.array([4,5])
m
```

```
array([4, 5])
```

```
n + m
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[35], line 1
----> 1 n + m

ValueError: operands could not be broadcast together with shapes (3,) (2,)
```

```
m = np.array([4,5,6])
m
```

```
array([4, 5, 6])
```

```
n + m
```

```
array([5, 7, 9])
```

Math Functions

Vectorized math

- np.sqrt / exp / log apply element-wise across the array.
- np.power(a, 2) squares every element.
- Trig helpers: np.sin / cos / tan.
- Plain operators (+ * /) are element-wise too.

math_ops.py

```
a = np.array([1.0, 4.0, 9.0, 16.0])

# Element-wise math
np.sqrt(a)      # [1. 2. 3. 4.]
np.exp(a)       # e ** a
np.log(a)       # natural log
np.power(a, 2)  # a squared

# Operators work element-wise too
a + 10          # [11. 14. 19. 26.]
a * a           # [1. 16. 81. 256.]
```

Transpose

```
n = np.array([[1,2,3], [4,5,6]])
n
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
n.T
```

```
array([[1, 4],
       [2, 5],
       [3, 6]])
```



Cast between types

```
n = np.array([[1,2,3], [4,5,6]])
n
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
n.dtype
```

```
dtype('int64')
```

```
n.astype('f')
```

```
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)
```



Aggregation

```
n = np.array([[1,2,3], [4,5,6]])
n
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
n.sum()
```

```
21
```

```
np.sum(n)
```

```
21
```

```
n.mean()
```

```
3.5
```

```
n.min()
```

```
1
```

```
n.max()
```

```
6
```

```
np.median(n)
```

```
3.5
```



Lesson 1.2: Indexing and Slicing

By single index

```
n[0]
```

```
0
```

```
n[11]
```

```
121
```

Slicing (By index range from start to end)

```
n[0:5]
```

```
array([ 0,  1,  4,  9, 16])
```

```
n[-3:]
```

```
array([100, 121, 144])
```

Slicing (By index range from start to end with step)

```
n[0:5:2]
```

```
array([ 0,  4, 16])
```

```
n[-5::1]
```

```
array([ 64,  81, 100, 121, 144])
```

```
n[-5::-1]
```

```
array([64, 49, 36, 25, 16,  9,  4,  1,  0])
```

NumPy Practice Exercises

Lists Into Arrays

1 List to Array

- Convert a plain Python list into an ndarray

```
mylist = [1, 2, 3]
x = np.___(mylist)

print(x)
```

Expected output:

[1] → [1 2 3]

2 Vector Dimensions

- Check the dimensions of y = [3, 4, 5]

```
y = np.array([3, 4, 5])

print(y.___)
```

Expected output:

[1] → (3,)

Operations & Math

SECTION 2 OF 5 · LESSON 1.1

Vectorized arithmetic with scalars and arrays, broadcasting, element-wise math functions, transpose, casting, and aggregation.

Scalar Arithmetic

1 Add & Subtract

- For n = [1, 2, 3], add 1
- and subtract 2, element-wise

```
n = np.array([1, 2, 3])
print(n ___ 1)
print(n ___ 2)

# hint: the same operators
# you'd use on plain numbers
```

Expected output:

[1] → [2 3 4]
[2] → [-1 0 1]

2 Divide & Raise a Power

- For the same n, divide by 3
- and square every element

```
n = np.array([1, 2, 3])
print(n ___ 3)
print(n ___ 2)

# hint: one splits values into
# thirds, the other raises a power
```

Expected output:

[1] → [0.33333333 0.66666667 1.   ]
[2] → [1 4 9]

Transpose & Type

7 Transpose a Matrix

- For n = [[1,2,3],[4,5,6]],
- flip its rows and columns

```
n = np.array([[1, 2, 3],
              [4, 5, 6]])
print(n.___)

# hint: a one-letter attribute,
# no parentheses needed
```

Expected output:

[[1 4]
[2 5]
[3 6]]

8 Inspect & Convert Type

- For n = [[1,2,3],[4,5,6]], print its stored type,
- then convert it to float32

```
n = np.array([[1, 2, 3],
              [4, 5, 6]])
print(n.___)
print(n.___(___))

# hint: one just reports the type,
# the other returns a converted copy of that type
```

Expected output:

[1] → int64
[2] → [[1. 2. 3.]
[3] → [4. 5. 6.]]

Totals & Stats

9 Total It Two Ways

- For the same n, add up all its values using both call styles

```
n = np.array([[1, 2, 3],
              [4, 5, 6]])
print(n.___())
print(np.___(n))

# hint: same operation, once as a
# method and once as a function
```

Expected output:

[1] → 21
[2] → 21

10 Four Quick Stats

- For the same n, compute its average, min, max, and middle value

```
n = np.array([[1, 2, 3],
              [4, 5, 6]])
print(n.___(), n.___(),
      n.___(), np.___(n))

# hint: four stats in one line -
# avg, smallest, largest, mid-point
```

Expected output:

[1] → 3.5 1 6 3.5

Indexing & Slicing

SECTION 3 OF 5 · LESSON 1.2

Pulling values out of an array by single index, and slicing ranges from the front, back, and with a step.

Single-Index Lookups

1 First Element

- Get the value at first element

```
n = np.array([0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100,
121, 144])

print(n[___])
```

Expected output:

[1] → 0

2 A Later Element

- Get the value at 2nd row, 2nd column

```
n = np.array([
        [0, 1],
        [ 4, 9]
])

print(n[___])
```

Expected output:

[1] → 9